Developer Contribution Guide: Anmol Thapliyal

Role: UI/UX Lead, Designer, & Turn-Based Gameplay Programmer
Key Areas: Boss Encounter Module, Turn-Based Combat Logic, Modular UI Systems

---

Executive Summary

This document outlines the specific technical and design contributions made by **Anmol Thapliyal** to the *Dungeon Escape CardBound* project. Anmol spearheaded the transition from real-time action to a strategic, turn-based card system for the climactic boss encounters. Additionally, as the UI/UX Lead, Anmol engineered a suite of highly polished, modular, and responsive overlay screens that govern the game's menus and inventory systems.

---

1. The Boss Encounter Module (`BossFightScreen.java`)

Anmol single-handedly designed and programmed the secondary game mode, drastically shifting the game's paradigm to a strategic card battler for the final encounters.

The Turn-Based Engine

Instead of relying on the real-time `GameWorld` logic, Anmol engineered a custom State Machine specifically for combat, defined by the `TurnState` enum:

- `PLAYER_TURN`: Awaiting player input to play cards.
- `BOSS_THINKING`: A timer-based delay where the boss calculates its next move.
- `BOSS_FLASHING_CARD`: A telegraphing phase giving the player visual feedback of the impending attack.
- `VICTORY` & `GAME_OVER`: Terminal states handling post-combat transitions.

Card System & Deck Management

Anmol implemented a fully functional deck-building mechanic within the `BossFightScreen`:

- **The `Card` Class:** A custom inner class that defines a card's `name`, `texture`, `cost` (energy), `value` (damage/heal/block amount), and `CardType` (`ATTACK`, `DEFEND`, `HEAL`, `BOSS_ATTACK`, `BOSS_ULTIMATE`).
- **Deck Mechanics:** Implemented standard card-game logic including a `drawPile`, `hand`, and `discardPile`. The logic (`startPlayerTurn()`) correctly shuffles the discard pile back into the draw pile when empty using `Collections.shuffle()`.
- **Energy Tracking:** Players are limited by an energy pool (`playerEnergy`), forcing strategic choices. Cards visually dim and become unplayable if the player lacks sufficient energy, calculated and rendered dynamically in the `render()` loop.

Advanced Combat Mechanics

- **Block System:** Implemented a temporary health buffer (`playerBlock`) that resets every turn, heavily inspired by modern roguelike deckbuilders (e.g., Slay the Spire).
- **Boss AI & Probability:** The boss doesn't just attack randomly; Anmol programmed a weighted probability system. During `BOSS_THINKING`, a random roll determines the attack: 50% Standard Attack, 35% Life Drain (custom logic that heals the boss), and 15% Ultimate Attack.

---

2. Modular UI/UX Architecture

As the UI/UX Lead, Anmol developed a highly responsive, immediate-mode GUI (IMGUI) overlay system. Rather than relying on heavy external UI frameworks, Anmol built performant, custom-drawn interfaces using raw LibGDX renderers.

The Inventory System (`InventoryOverlay.java`)

- **Grid Mathematics:** Programmed a dynamic grid system (`GRID_COLS`, `GRID_ROWS`) that automatically calculates slot positions and scales them based on the current window size (using `Viewport`).
- **Input Handling:** Supported both mouse hovering/clicking and keyboard navigation (WASD/Arrow keys) with seamless wrapping logic `(selectedCol + 1) % GRID_COLS`.
- **Visual Polish:** Implemented color-coded states (backgrounds, active slots, glowing selection borders) and dynamically rendered item quantities with scaled `BitmapFont` text over item icons.

Interactive Menus (`PauseOverlay.java` & `SettingsOverlay.java`)

- **State-Driven Transitions:** Built the `SettingsOverlay` with a custom `State` enum (`TRANSITION_IN`, `ACTIVE`, `TRANSITION_OUT`) and an `Interpolation.pow2Out` math function to create smooth, professional slide-in/fade-in animations.
- **Sine Wave Animations:** Wrote mathematical functions using `MathUtils.sin(animTime * 7f)` to create a pulsing "breathing" effect on the currently selected menu item, drastically improving user feedback.
- **Text Shadowing:** Engineered a custom `drawTextWithShadow` technique, rendering text twice (once offset in black, once centered in white) to ensure menu text is perfectly legible regardless of the background color.

---

3. Technical Stack & Libraries Used

Anmol's work heavily utilized the core rendering pipeline of **LibGDX**:

- **`SpriteBatch`:** Used extensively in `BossFightScreen` to render the boss sprites, player sprites, background art, and dynamically positioned playing cards.
- **`ShapeRenderer`:** The backbone of Anmol's UI. Used to draw health bars (with layered block indicators), inventory grid slots, transparent overlays, and colored borders using `ShapeType.Filled` and `ShapeType.Line`.
- **`BitmapFont` & `GlyphLayout`:** Used to render combat logs, boss names, and dynamic health/energy text. `GlyphLayout` was utilized to calculate text width in real-time `(viewport.getWorldWidth() - glyphLayout.width) / 2f` to ensure perfect centering.
- **`Viewport` (`FitViewport`):** Ensured that all UI elements and the card battle screen scale flawlessly regardless of whether the user plays in 800x600, 1080p, or Fullscreen mode.
- **`Vector3` & Unprojection:** Used `viewport.unproject(touch)` to map raw mouse clicks on the screen directly to the virtual bounds of the playing cards and menu buttons.

Conclusion

Anmol Thapliyal's contributions form the strategic core and the visual presentation layer of the game. By coding a completely distinct tum-based combat loop and a mathematically precise UI framework from scratch, Anmol demonstrated strong command over state machines, game loop rendering, user interaction, and Object-Oriented design.